一、 操作系统与内核调度

Q1: 进程与线程的底层区别是什么？

- **资源维度**：进程是**资源分配**的最小单位（拥有独立的虚拟地址空间、页表、文件描述符）；线程是**CPU调度**的最小单位（共享进程的堆、代码段、全局变量，仅私有栈和寄存器）。
- **切换开销（核心点）**：
  - **进程切换**：涉及虚拟内存空间切换，导致 **TLB (页表缓冲) 全部失效**，L1/L2 Cache 变冷，开销极大（微秒级甚至更高）。
  - **线程切换**：仅需保存/恢复**寄存器上下文 (Register Context)**（PC, SP, 通用寄存器），依然共享 TLB 和 Cache，开销极小（约 1~2 微秒）。

Q3: 为什么互斥锁 (Mutex) 的开销比自旋锁 (Spinlock) 大？

- **Mutex (std::mutex)**：底层基于 Linux 的 futex 系统调用。当无法获取锁时，线程被内核挂起 (Sleep)，从用户态陷入内核态，触发**上下文切换**，移除出 CPU 运行队列。
- **Spinlock**：纯用户态操作。利用 CPU 原子指令 (CAS) 在循环中忙等待。没有系统调用，没有上下文切换。
- **选择**：临界区极短（< 上下文切换耗时）用 Spinlock；临界区长或不可预测用 Mutex。

二、 硬件架构与内存模型

Q4: 什么是伪共享 (False Sharing) 及其底层原理？

- **原理**：CPU 缓存以 **Cache Line (通常64字节)** 为单位进行读写。若两个不同线程的变量（如 A 和 B）位于同一个 Cache Line 中，当核心 1 修改 A 时，根据 **MESI 缓存一致性协议**，必须标记该 Cache Line 为 Invalid。
- **后果**：核心 2 读取 B 时，发现缓存失效，被迫重新从 L3 或内存加载数据，导致核心间频繁争抢总线所有权，性能剧降。
- **解决**：使用 alignas(64) 进行内存对齐，强制隔离变量到不同的 Cache Line。

Q5: 解释 CAS (Compare-And-Swap) 与 ABA 问题。

- **CAS**：对应 x86 指令 lock cmpxchg。它是**硬件提供的原子原语**。逻辑为：*“如果内存值等于预期值 A，则修改为 B，否则失败”*。
- **ABA 问题**：线程 1 准备把 A 改为 B，此时线程 2 将 A 改为 C 又改回 A。线程 1 执行 CAS 时发现值还是 A，认为未被修改，从而错误执行。
- **解决**：引入**版本号 (Versioning)**。将 value 变为 pair<value, version>，每次修改不仅更新值，还让 version + 1。

Q6: volatile 关键字在多线程中有什么用？

- **答案**：**没用**（用于同步时）。
- **底层**：volatile 仅禁止**编译器**对该变量的读写优化（强制每次读写内存，不利用寄存器缓存）。但它**不禁止 CPU 乱序执行**，也**不保证原子性**。

- **正确做法**：必须使用 std::atomic，它既保证原子性，又通过内存屏障 (Memory Fence) 禁止指令重排。

Q7: 内存序 (Memory Order) 中的 Acquire 和 Release 是什么意思？

- **指令重排**：为了优化，编译器和 CPU 可能打乱指令执行顺序。
- **Acquire (获得/读)**：保证该操作**之后**的读写指令，不会被重排到该操作之前。（防止“还没拿到锁，就先读取了数据”）。
- **Release (释放/写)**：保证该操作**之前**的读写指令，不会被重排到该操作之后。（防止“锁释放了，数据还没写进去”）。
- **作用**：二者配合建立 **Happens-Before** 关系，实现无锁编程中的同步。

三、 C++ 并发编程陷阱

Q8: std::condition_variable 为什么必须配合 while 循环使用？

- **虚假唤醒 (Spurious Wakeup)**：底层系统（如 Linux pthread_cond_wait）可能在因信号中断等原因，在未收到 notify 时返回。
- **竞态条件 (Race Condition)**：即使被 notify 唤醒，从“唤醒”到“重新拿锁”之间，可能有其他线程抢先获取锁并修改了状态（如取走了队列元素）。
- **结论**：必须循环检查谓词条件 (predicate)，确保逻辑正确。

Q9: std::shared_ptr 是线程安全的吗？

- **结论**：**部分安全**。
- **安全部分**：**引用计数 (Control Block)** 的操作是原子的（底层用 atomic），多线程拷贝/销毁 shared_ptr 本身不会导致内存泄漏或 Double Free。
- **不安全部分**：**管理的对象 (Managed Object)** 不是线程安全的。如果多线程同时读写同一个 shared_ptr 指向的对象，必须手动加锁。

Q10: 为什么单例模式 (Singleton) 推荐使用 "Meyers' Singleton"？

- **代码**：static T& getInstance() { static T instance; return instance; }
- **原理**：**C++11 Magic Statics**。标准规定：局部静态变量的初始化是线程安全的。编译器会自动插入类似 call_once 的保护代码（通常是一个 hidden flag 和 lock），防止多线程并发初始化时构造多次。无需手写复杂的 Double-Checked Locking。

Q11: 什么是死锁 (Deadlock) 的四个必要条件？

1. **互斥 (Mutual Exclusion)**：资源独占。
2. **占有并等待 (Hold and Wait)**：拿了一个锁，还在等另一个，且不释放手里的。
3. **不可抢占 (No Preemption)**：由于资源无法被强行剥夺。
4. **循环等待 (Circular Wait)**：A 等 B，B 等 A。

- **工程解法**：破坏“循环等待”——严格规定**锁序 (Lock Hierarchy)**（例如所有线程必须先锁 A 再锁 B）。